Genius Kids Academy — Full System Report

Nov 4, 2026 · @homos

Executive Summary

This document is the full technical and functional specification for the Genius Kids Academy ERP — a custom web application that will replace the current Excel and paper-based system across all 13 branches.

The system covers two branch types: **academies** (workshops, levels, evaluation, progression) and **kindergartens** (age groups, daily logs, monthly reports, communication books). It manages the full lifecycle of students and staff, tracks every dinar that enters or leaves each branch, and gives parents a live window into their child's attendance and progress.

Scope in numbers: ~13 branches · ~5,200 students · 2 branch types · 9 functional modules · 1 shared PostgreSQL database · 3 years of historical data to migrate.

What it replaces: one Excel workbook per branch (student lists, payment tracking, timetables, expense logs) and paper books (Soroban follow-up books, kindergarten communication books, class journals).

What it adds that Excel cannot do: real-time absence alerts to parents, duplicate-student prevention across branches, a cash balance that is always current, one login for a parent with children in several branches, and a full audit trail of every financial change.

Business Context

Genius Kids Academy is a private school group in Jijel, Algeria, with 13 branches serving children aged 3 to 14. Each branch is either a kindergarten or an academy, and people (students and staff) can move between branches.

Academy model. The academy teaches *workshops* (Soroban mental arithmetic, robotics, French, English, AI, Rubik's cube, Quran, horse riding, swimming, support lessons, a summer club and competitions). A workshop has one or more *courses*; a course is a level and has one book. Students move through books in strict order, cannot skip a book, cannot take two books of the same workshop at once, and repeat a book if they fail its

tests. Each workshop defines its own way of evaluating: Soroban uses timed operations and three tests with an 80/100 pass mark; some groups are tested orally with right/wrong answers; robotics uses projects; horse riding uses challenges instead of a book.

Kindergarten model. Children are split by age, then into groups of the same age. Each child follows a program, owns a numbered book whose content depends on age and level, and gets a daily report rated A/B/C (good, average, weak) on meals, sleep, interaction, crying and clothes. Teachers also write a monthly report that administration approves before parents can see it. An orthophonist can leave notes, and parents can send simple notes that staff answer.

Money. Payment is cash only. Academy fees are paid per level, upfront or in installments. Kindergarten fees are paid monthly or yearly, and yearly subscribers get a discount. Registration is paid every year. The receptionist can set the price per customer, so discounts are dynamic. Payroll is handled outside the system and is out of scope.

Who uses it: administration, receptionists, teachers and nannies, specialists, other staff (cleaning, maintenance, cooks), students (including under-14s, who log in themselves) and parents.

Current tools and what replaces them

Today	What it holds	Replaced by
Excel per branch: student sheets per workshop	Names, level, trainer, up to 4 installments, receipt numbers	Enrollment, invoices, installments, payments
Excel: group timetable grid	Day, time slot, room, level, trainer	Groups, weekly slots, sessions
Excel: daily cash sheets	Income, expenses, handovers, remaining cash	Cash ledger and daily summary view
Excel: kitchen sheets	Bread and meat purchases	Expense categories only
Soroban follow-up book	Group list, entry assessment, speed race, support sessions	Evaluation templates and results, make-up sessions
Kindergarten communication book	Menu, child info, daily report	Daily logs, monthly reports, parent notes

Today	What it holds	Replaced by
Class journal	Daily attendance grid, behavior and eating problems	Kindergarten attendance and incident log

System Scope and Modules

The system is organized into nine modules that share one person registry, one set of branches and one finance ledger.

#	Module	What it covers	Applies to
1	Identity and access	Accounts, dynamic roles built from a fixed permission catalogue, role assignment per branch, audit log	All
2	Organization	Branches, academic years, rooms and room equipment	All
3	People and family	One record per person, guardians, authorized pickup list, medical info, documents and photos	All
4	Staff and HR	Staff profiles, job titles, branch assignments, daily staff attendance, online applications and interviews	All
5	Academy	Workshops, courses and books, pass rules, groups, weekly timetable, sessions, enrollment, attendance, make-up sessions, dynamic evaluations, online books and homework	Academies
6	Kindergarten	Age bands, groups, daily attendance, daily logs, monthly reports with approval, incident log, orthophonist notes, numbered books, weekly menu	Kindergartens
7	Communication	Parent notes (switchable by time), email, SMS, WhatsApp and Telegram notifications	All

#	Module	What it covers	Applies to
8	Finance	Price plans, invoices, installments, cash payments and receipts, cash box per branch, expenses, handovers, daily summary	All
9	Data migration	Import of Excel history with a review step	All

Out of scope

- Payroll, salaries, CNAS, CASNOS and taxes (handled externally)
- Payment methods other than cash (the structure allows adding them later)
- Consumption tracking in the kitchen (only expenses by category are recorded)
- A log of who picked up each child each day (only the authorization list is kept)
- Automated timetable generation (timetables are built by staff, and the system blocks clashes)

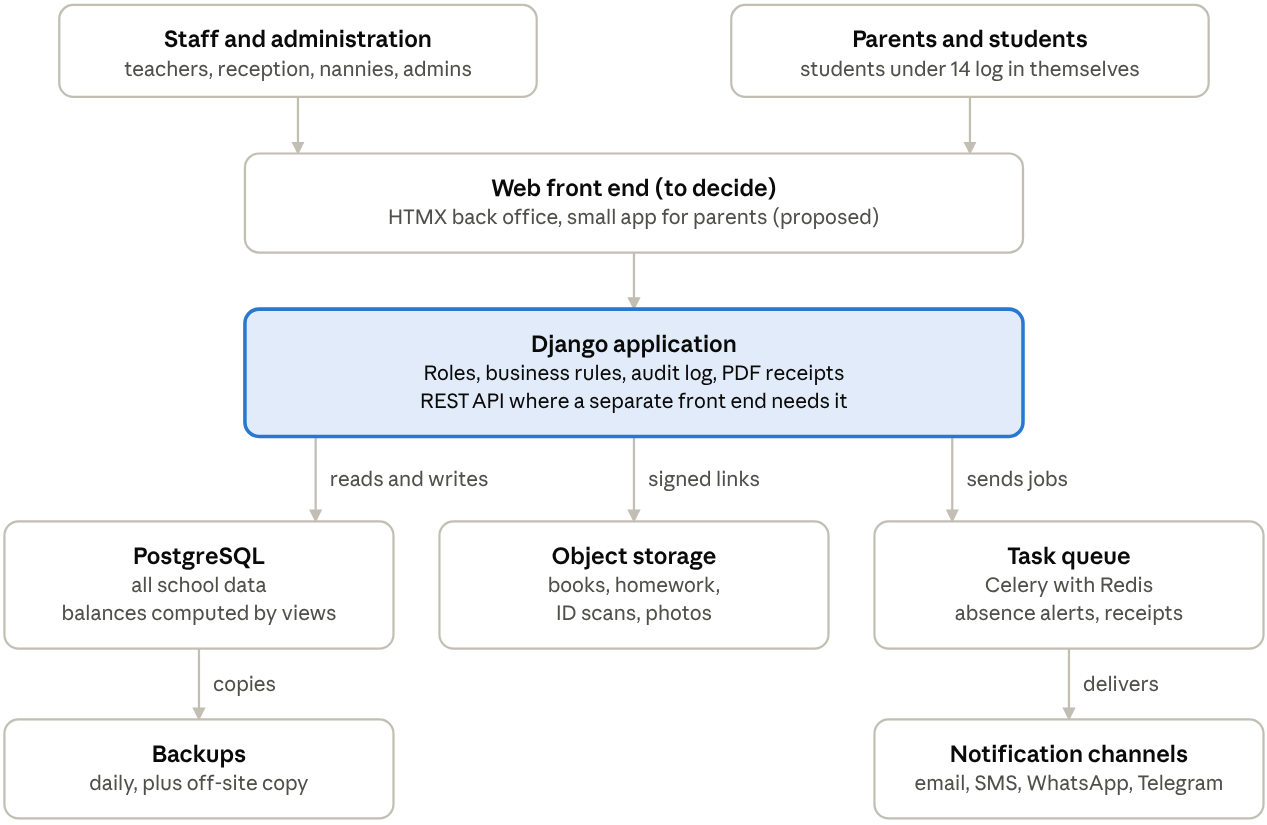
Later phases (not decided)

- Fingerprint device integration for staff attendance (the attendance table already has a source field for it)
- Legal invoice fields, if the school ever needs them
- Mobile app for parents, if a web app proves insufficient

Technical Architecture

The proposed stack keeps everything in one Django application: staff and parents reach it through the web, and it alone talks to the database, the file store and the notification queue.

One Django backend sits between users, data, files and notifications



system architecture · 1 backend, 3 services, 2 user groups

These are recommendations for the lead developer to confirm; none is final.

Layer	Proposal	Why
Backend	Django, with Django REST Framework only where an API is needed	An ERP with many tables, permissions, migrations, admin screens and file handling gets the most from Django's built-in parts. FastAPI would mean rebuilding auth, admin and migrations.
Front end	Django templates with HTMX and Alpine.js for back office; a small React or PWA for parents and students if a richer app is wanted	Faster to build and maintain for a small team. A mobile app later would use the API.

Layer	Proposal	Why
Database	PostgreSQL	Strong constraints, exclusion constraints for timetable clashes, partial unique indexes. About 5,200 students is a light load.
Background jobs	Celery with Redis (or a lighter option such as Dramatiq or Django-Q)	Email, SMS, WhatsApp and Telegram messages, absence alerts and report generation.
Files	MinIO if hosted on site, or S3 / Cloudflare R2 in the cloud	A second database is not needed; keep only metadata in PostgreSQL.
PDF	WeasyPrint (HTML to PDF)	Invoices, receipts and report cards with right-to-left Arabic support.
Notifications	One queue table and one sender interface per channel	New channels can be added without touching business code.

Database Design

The first-pass schema is written (`school_erp_schema_v1.sql`): about 75 tables, 3 views and seed data, loaded and tested on PostgreSQL 16 with fake data. It is the specification the developers translate into Django models.

Conventions. Table names carry a module prefix (`core_`, `org_`, `iam_`, `hr_`, `acad_`, `kg_`, `comm_`, `fin_`, `mig_`) so they map directly to Django apps. Primary keys are bigint identities, money is `NUMERIC(12,2)` in dinars, timestamps are `timestamptz`, and weekdays follow ISO numbering (1 = Monday).

Core tables by module

Module	Main tables
Core	`core_person`, `core_student`, `core_student_medical`, `core_guardian_link`, `core_pickup_authorization`, `core_file`, `core_person_document`, `core_audit_log`

Module	Main tables
Organization	`org_branch`, `org_academic_year`, `org_room`, `org_room_equipment`
Access	`iam_account`, `iam_permission`, `iam_role`, `iam_role_permission`, `iam_person_role`
HR	`hr_position`, `hr_staff`, `hr_staff_assignment`, `hr_staff_attendance`, `hr_staff_application`
Academy	`acad_workshop`, `acad_course`, `acad_course_rule`, `acad_group`, `acad_group_slot`, `acad_enrollment`, `acad_session`, `acad_attendance`, `acad_makeup`, `acad_eval_template`, `acad_eval_field`, `acad_eval_event`, `acad_eval_result`, `acad_eval_result_value`, `acad_material`
Kindergarten	`kg_age_band`, `kg_group`, `kg_enrollment`, `kg_attendance`, `kg_daily_log`, `kg_monthly_report`, `kg_incident`, `kg_specialist_note`, `kg_child_book`, `kg_menu_entry`
Communication	`comm_note`, `comm_note_policy`, `comm_notification`
Finance	`fin_fee_plan`, `fin_invoice`, `fin_invoice_line`, `fin_installment`, `fin_payment`, `fin_payment_allocation`, `fin_cash_box`, `fin_cash_entry`, `fin_expense_category`
Migration	`mig_import_batch`, `mig_import_row`, `mig_legacy_ref`

Rules the database itself enforces

These were tested and fail correctly when broken:

- A level cannot be started unless the previous level was passed, or an admin records a placement note.
- A student has only one open book per workshop.

- A room or a trainer cannot be double-booked, in the weekly timetable or on a specific date.
- A group cannot use a room from another branch.
- An installment cannot be overpaid, and a payment cannot be allocated beyond its amount.
- Payments cannot be deleted, only voided with a reason.
- A price different from the list price needs a reason and the name of the person who set it.
- The person who reviews a test result must differ from the one who corrected it; the same rule applies to kindergarten monthly reports.
- Only one academic year can be current, and years cannot overlap.

Derived, never stored

Cash balances, daily income and expense summaries, and invoice remaining balances are computed by views from payments and cash entries, so they cannot drift out of sync.

Key Design Decisions

These choices shape the whole system. Each can be revisited, but changing one later costs more than deciding it now.

Decision	Choice	Reason and trade-off
Tenancy	Single organization; operational tables carry `branch_id`	Simpler than multi-tenant. Selling the system to other schools would need rework.
People	One `core_person` for everyone; roles and profiles attach to it	A teacher who is also a parent is one record. One account works across all branches.
Evaluation	Per-workshop templates with typed fields; pass rules stored as data	Soroban times, oral right/wrong and robotics projects need no special code. Reports on values use joins instead of one wide table.

Decision	Choice	Reason and trade-off
Progression	Enforced in the database (sequence trigger, one open book per workshop)	Rules hold even if a screen has a bug. A placement note allows exceptions for new students.
Roles	Admins create roles from a fixed permission catalogue; assignment is person + role + branch	Admins cannot invent permissions that the code does not check. A role can apply to one branch or all.
Job title vs role	Job titles (cook, cleaner, orthophonist) are separate from permission roles	Staff who never log in still get attendance and assignments.
Kindergarten model	Own tables for groups, daily attendance, logs and reports	The kindergarten works per day, the academy per session. Forcing one model onto both would distort both.
Make-up sessions	Ordinary sessions with their own attendee list; one `acad_makeup` table covers both absences and failed tests	Make-ups are not tied to the original group or room.
Pricing	List price per branch, workshop, level and year; the receptionist can override the charged price with a reason	Matches how the school works today while keeping every discount traceable.
Money	Payment is the receipt; balances come from views; voids instead of deletes	Cash cannot silently change. A payment may exceed its allocations, which records an advance.
Files	Object storage (MinIO, S3 or R2) with metadata in PostgreSQL; short-lived signed links	Keeps the database small and backups fast. Access is checked per student and course.
Account activation	Parent and student accounts stay pending until registration and first-level fees are paid; implemented in application logic	A database trigger would hide business rules. Activation can run automatically on payment.

Decision	Choice	Reason and trade-off
History	Records are archived or voided, not deleted; changes go to an audit log	Required for grades and money. The audit log must cover grade edits and price overrides.

Functional Requirements by Module

Each requirement below comes from the requirements interviews, the Excel files or the paper books. Items marked *(to confirm)* are my reading of a photo or an assumption.

1. Identity and access

- Anyone can create an account online; it stays pending until the required fees are paid, then it is activated. This applies to both the parent and the student.
- Students under 14 log in themselves; parents log in separately and see their children.
- Admins create roles by picking permissions from a fixed catalogue, and assign roles per person and per branch (or all branches). One person can hold several roles.
- Every sensitive action (grade change, price override, payment void, medical data view) is written to the audit log.

2. Organization

- Manage 13 branches, each a kindergarten or an academy, with rooms, room capacity and equipment per room.
- Academic years start at the beginning of the school year; a late student can enroll if a place is available.

3. People and family

- Student data: Arabic and Latin names, birth date, contact details, school grade, previous schools, medical information, special needs, documents and photos.
- Link each student to father, mother or guardian, with phone numbers, emergency contact and custody flags.
- Parents set in the app who may pick up the child, either a person already in the system or a one-off person. Reception checks the ID at the desk. No pickup log is kept.

4. Staff and HR

- Staff profiles with job titles (teaching, administrative, cleaning, maintenance, cooks, orthophonist and others) and assignments that can change between branches.
- Staff attendance is marked once a day by administration; a fingerprint device can feed the same table later.
- Staff apply online and are invited by email for an interview; the application status is tracked.

5. Academy

- Define workshops, their courses (levels) in order, the book for each course and a pass rule per course (for Soroban: 3 tests, 80 out of 100 each).
- Create groups per branch and year with trainer and capacity; build the weekly timetable by group size and room capacity. Hours are mostly fixed, rooms may change for a session.
- Mark attendance per session. Administration calls parents about absences and lateness, and records justifications received by phone.
- An absence creates a make-up session owed to the student. A failed test creates a make-up session for that test only.
- Evaluations are dynamic per workshop: timed operations, oral right/wrong, projects, challenge milestones, scores. The teacher corrects and administration reviews before the result is published.
- A student who passes all required tests moves to the next book; a student who fails repeats the book.
- Students download online books and homework only for the courses they are enrolled in.

6. Kindergarten

- Group children by age band, then into groups of the same age; each child has a numbered book whose content follows age and level.
- Daily attendance per child.
- Daily log rated A, B or C on a fixed set of items: breakfast, lunch, snack, sleep, study, interaction, crying, clothes change, plus a free note *(to confirm items)*.
- Monthly report with the same items and a note; administration approves before parents see it.

- Incident log for behavior, sleep, eating and learning problems *(to confirm categories)*.
- Orthophonist notes, visible to staff or also to parents.
- Child information page filled by parents (food, sleep, clothing, illness, temperament) and a weekly rotating menu shown to parents.

7. Communication

- Parents send a simple note per child to administration or the teacher; staff answer once. Administration switches this feature on or off, or limits it to time windows.
- Notifications go out by email, SMS, WhatsApp and Telegram through one queue: absence alerts, payment receipts, report approval, interview invitations.

8. Finance

- Price plans per branch, workshop, level and year; kindergarten plans for monthly and yearly billing; registration billed yearly.
- Invoices with lines, an installment schedule (one installment for upfront payment, several for split payment) and PDF output.
- Cash payments produce numbered receipts per branch and year without gaps; a payment can be voided but never deleted.
- One cash box per branch records expenses by category and handovers of cash to the owner or head office.
- The daily summary shows income, expenses, handovers and the running balance for each branch.

9. Data migration

- Load the Excel history into staging, review rows the importer could not match, then convert them to real records with a link back to the source sheet and row.

Non-Functional Requirements

The load is small (about 5,200 students across 13 branches), so the priorities are correctness, privacy and safe backups rather than raw performance.

Area	Requirement	Status
Scale	About 5,200 students and kids, 13 branches; one PostgreSQL server is enough at this size	Proposed
Languages	Arabic with right-to-left layout as the main language; French and Latin-script names stored for lists and certificates	To design
Privacy	Children's data, medical data and ID documents need restricted permissions; check Algerian Law 18-07 and its registration duties with a lawyer	Open
Audit	Log grade changes, price overrides, payment voids and views of medical data; keep history of past records	Decided
Backups	Automated daily database backups plus off-site copies from day one; files backed up separately	Proposed
Hosting	Not yet decided: a European provider (cheap, higher latency from Algeria) or a local Algerian datacenter (lower latency, usually pricier). Starting size: 4 vCPU, 8 GB RAM	Open
Availability	Staff work during opening hours; short maintenance windows outside them are acceptable. Offline use is not required	Assumed
Peak load	Start of the school year (enrollment and payments) and the end of each stage (test results)	Assumed
Files	Private storage; books and homework served through short-lived signed links, with view-only or watermarking if books are licensed	Proposed
Notifications	One queue with one sender per channel (email, SMS through a local gateway, WhatsApp Business API, Telegram bot)	Proposed
Documents	Invoices, receipts and report cards as PDF with correct Arabic rendering	Proposed

Migration Plan

Everything that exists in Excel, including grades where they exist, will be loaded into the new database. At least three years of history are involved. The team does not yet have the real data files, so the plan is designed around the structure of the sample files received.

1. **Collect.** Obtain the real workbooks for every branch, the list of all 13 branches with their type, and any file holding grades or Soroban test times. None of these are available yet.
2. **Stage.** Load each sheet row as raw data into `mig_import_row`, one batch per file and sheet.
3. **Clean and match.** Normalize names, split family and given names, unify level and group labels, and detect people who appear in several sheets or branches.
4. **Review.** Staff review the rows flagged as unmatched or ambiguous before anything becomes a real record.
5. **Convert.** Create people, enrollments, invoices, installments and payments, keeping a link to the source file, sheet and row in `mig_legacy_ref`.
6. **Reconcile.** Compare totals per branch and per month against the Excel cash sheets before go-live.

Data problems already seen in the samples

- Names are written as one field and the order of family name and given name is inconsistent, so splitting needs a review queue.
- Level and group labels are inconsistent (for example `p.section 2` and `p section 2`).
- Some hidden sheets contain `#REF!` errors and likely obsolete prices.
- A January date in the kindergarten expense sheet is in 2025 while the neighboring months are in 2026.
- The sample student and payment rows are mostly blank, so real data quality is unknown.

Risks

- Payment history is the hardest part to trust: receipt numbers, installment dates and partial payments are typed by hand.
- Grades kept on paper cannot be migrated automatically; the team must decide whether to key them in or archive scans as files.

Open Questions and Next Steps

The design is complete enough to start building the core, but these questions remain and a few block specific parts.

Open questions for the school

- [ ] Confirm the kindergarten daily report items and the four incident categories read from the photos.
- [ ] Give the exact kindergarten age bands and group names (Bébé, Petite, Moyenne, Grande and their sub-numbers).
- [ ] Decide whether the preparatory class (تحضيري) behaves like the kindergarten or like an academy workshop.
- [ ] Confirm that handovers of cash to the owner or head office must be recorded.
- [ ] Decide whether a student who repeats a level pays again.
- [ ] Decide who may override prices and whether a limit or a daily override report is needed.
- [ ] List all 13 branches with their type, and the workshops each one offers.
- [ ] Provide the real Excel files and any grade records for the migration.
- [ ] Read the games table on page 04 of the Soroban book, which was not legible in the photo.

Open questions for the developers

- [ ] Choose hosting: European provider or local Algerian datacenter.
- [ ] Choose the frontend: HTMX for back-office screens and a lighter app for parents and students, or a full React front end on a REST API.
- [ ] Pick the SMS gateway, and decide on WhatsApp Business API and Telegram bot accounts.
- [ ] Review the schema file and flag any table the framework makes awkward.

Proposed next steps

1. Developers review `school_erp_schema_v1.sql` and translate it into Django models, including the exclusion and partial unique constraints.
2. Write the permission catalogue and the default roles (receptionist, teacher, nanny, administrator, director).

3. Build the core first: organization, people, accounts, roles and audit log.
4. Build finance next, since it replaces the cash sheets and is the part the school checks daily.
5. Build the academy module, then the kindergarten module.
6. Run migration rehearsals on real data once it is available, and reconcile cash totals before go-live.